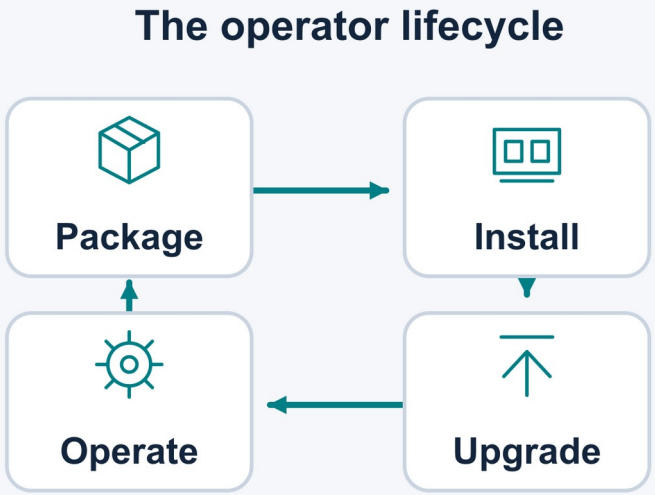
The Next Era of Kubernetes Operator Lifecycle Management



KubeCon + CloudNativeCon North America 2026

Presentation team

Rashmi

Joe

Jordan

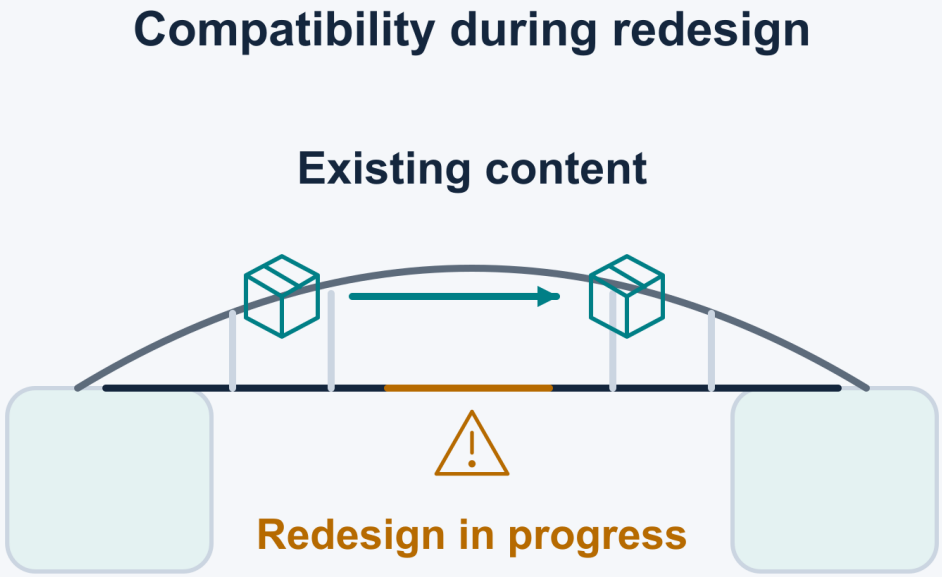
Compatibility and lifecycle foundations

FOUNDATIONAL WORK

OLM evolution builds on existing packages and operational experience.

- OLMv0 established lifecycle management
- Existing registry+v1 content shapes compatibility
- Operational experience informs OLMv1

Existing content and users shape lifecycle implementation.

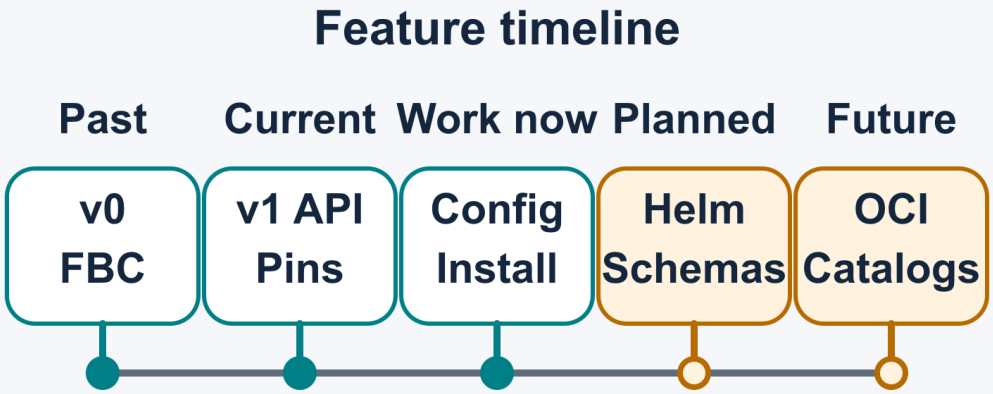


OLM: past, current and future

EVOLUTION / PLANNED / FUTURE WORK

OLM feature evolution

- Past: OLMv0, flawed namespace-local CRD visibility assumptions, SQLite, FBC
- Current: OLMv1, ClusterExtension, pins, webhooks
- Ongoing: rollouts, config, install, migration



Feature sequence; ordering to refine

Planned: helm+v2 and v2 schemas. Future: OCI catalog storage.

Install a bundle directly

IN ACTIVE DEVELOPMENT / CONTROLLED BY FEATURE-GATE

Select a registry+v1 bundle by its OCI image reference.

- Use sourceType: OCIImage
- Set source.ociImage.ref
- Bypass catalog resolution

Proposed in open PR #2907; controlled by BoxcutterRuntime.

Direct bundle installation



Direct source: OCIImage

Content type: registry+v1

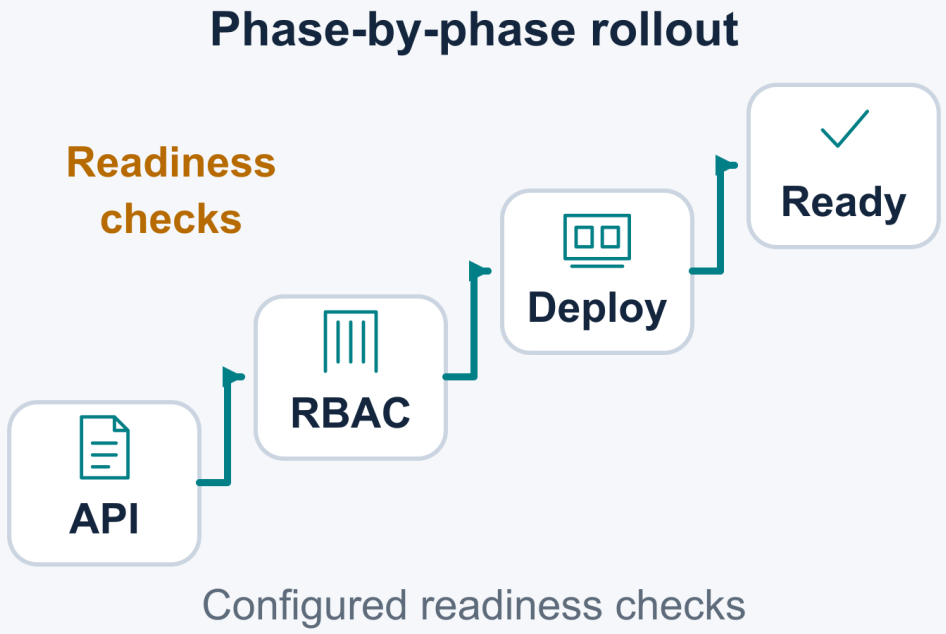
Inspect phased resource rollouts

IN ACTIVE DEVELOPMENT / CONTROLLED BY FEATURE-GATE

Ordered phases expose resource application and readiness progress.

- Apply resource groups in phase order
- Wait for applicable readiness checks
- Inspect revision content and ownership changes

Readiness checks show which phase is waiting.

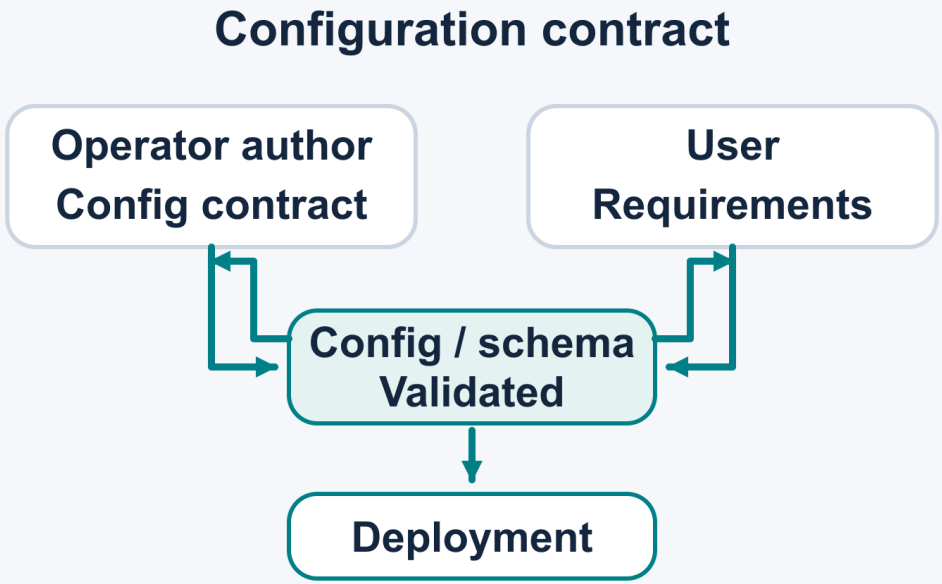


User control of extension deployments

IN ACTIVE DEVELOPMENT / CONTROLLED BY FEATURE-GATE

Flexible user-defined deployment configuration.

- Environment variables and resource settings
- Placement, volumes and deployment metadata
- Operator-provided configuration and schemas



OLM applies user choices through operator configuration.

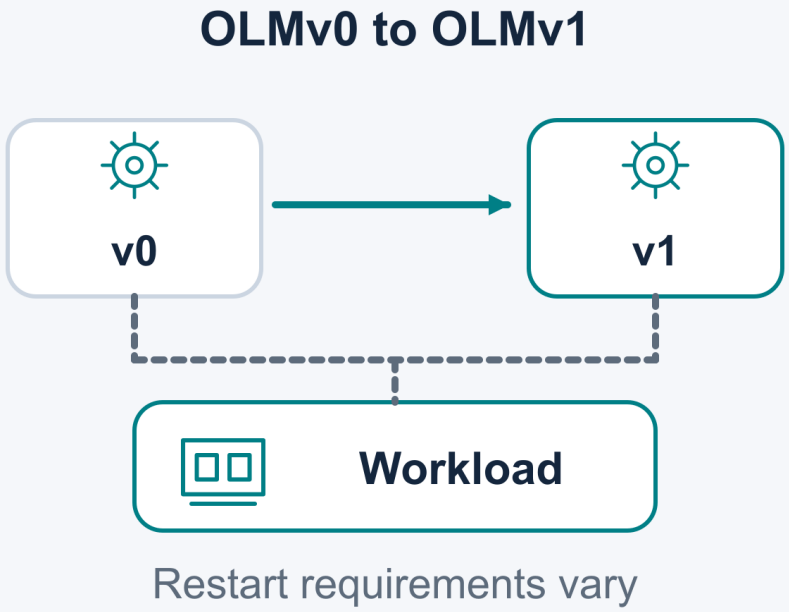
OLMv0 → OLMv1 migration

IN ACTIVE DEVELOPMENT

Move operator management from OLMv0 to OLMv1.

- Prepare v1 catalogs from v0 CatalogSources
- Check and preview operator migrations
- Migration library with example binaries

Structured output supports automation and models.



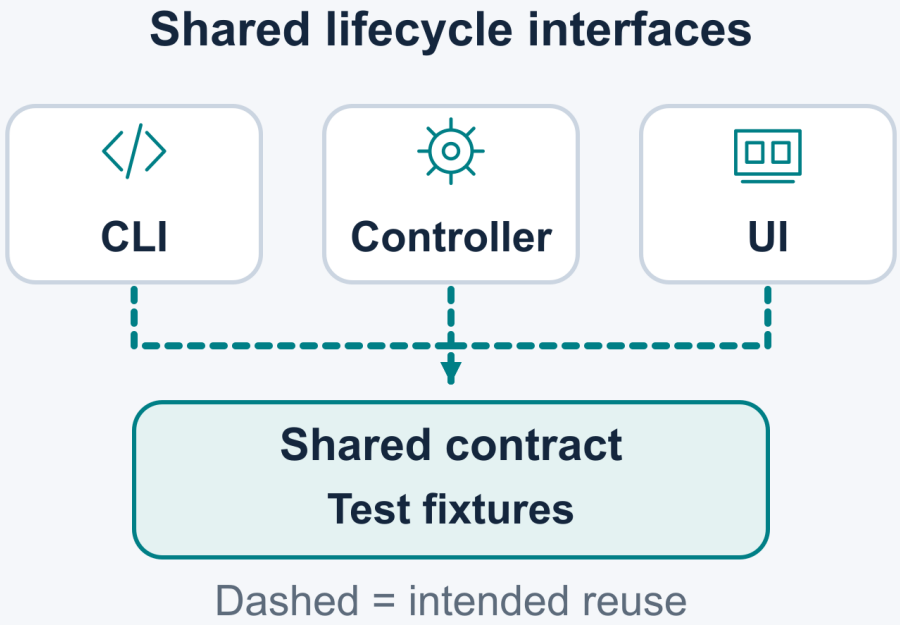
Shared lifecycle logic in library-olm

IN ACTIVE DEVELOPMENT

Reuse lifecycle logic through a library with explicit contracts.

- Share migration checks and workflow logic
- Exercise current development examples
- Record expected results in reusable fixtures

Examples and fixtures make supported behavior concrete.



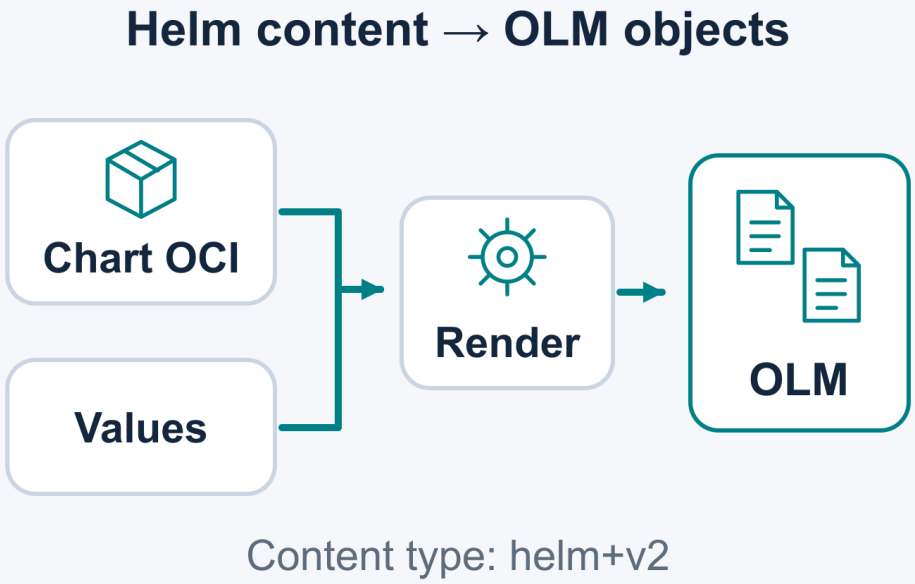
Planned Helm content support

PLANNED / helm+v2

The rendering contract targets deterministic chart behavior.

- Chart and values produce managed manifests
- Allow-listed features support reproducible rendering
- helm+v2 identifies the content representation

Standard chart artifacts use a defined rendering contract.



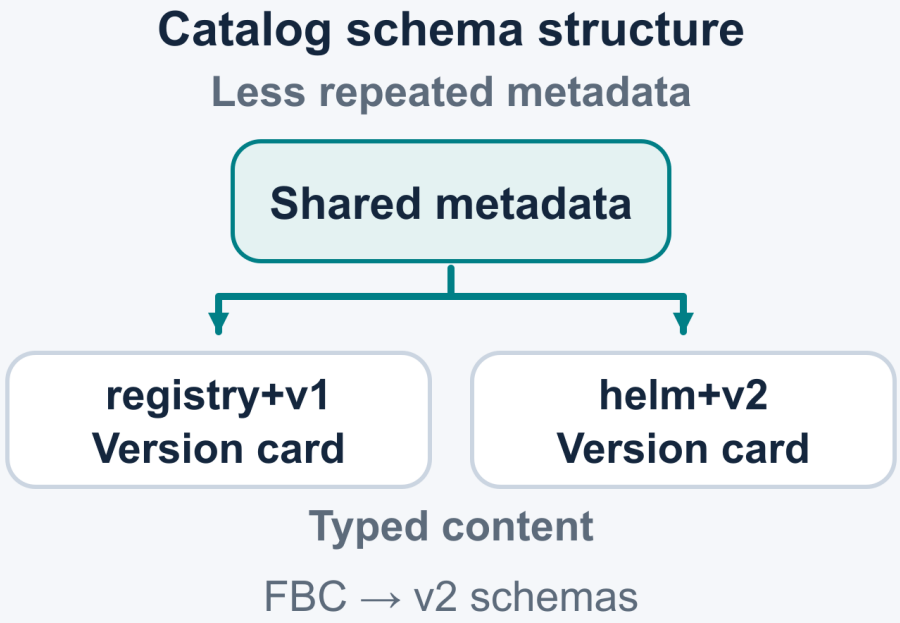
Catalog performance and content types

PLANNED / FUTURE WORK

Design for lower catalog-processing cost and additional content types.

- Lower duplicated CSV metadata
- Direct version fields reduce property traversal
- Typed registry+v1 and helm+v2 content

File-based catalogs remain the established foundation.



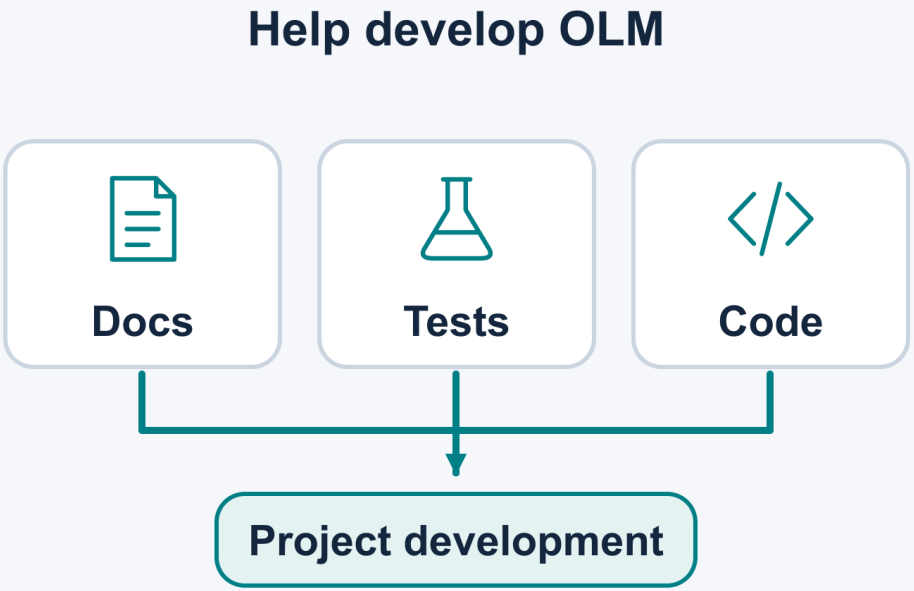
Contribute to Operator Framework

PROJECT DEVELOPMENT

Help develop the project through documentation, code and test cases.

- Improve documentation and examples
- Contribute code and focused tests
- Add migration, chart and schema fixtures

Start with a reproducible case and a clear expected result.

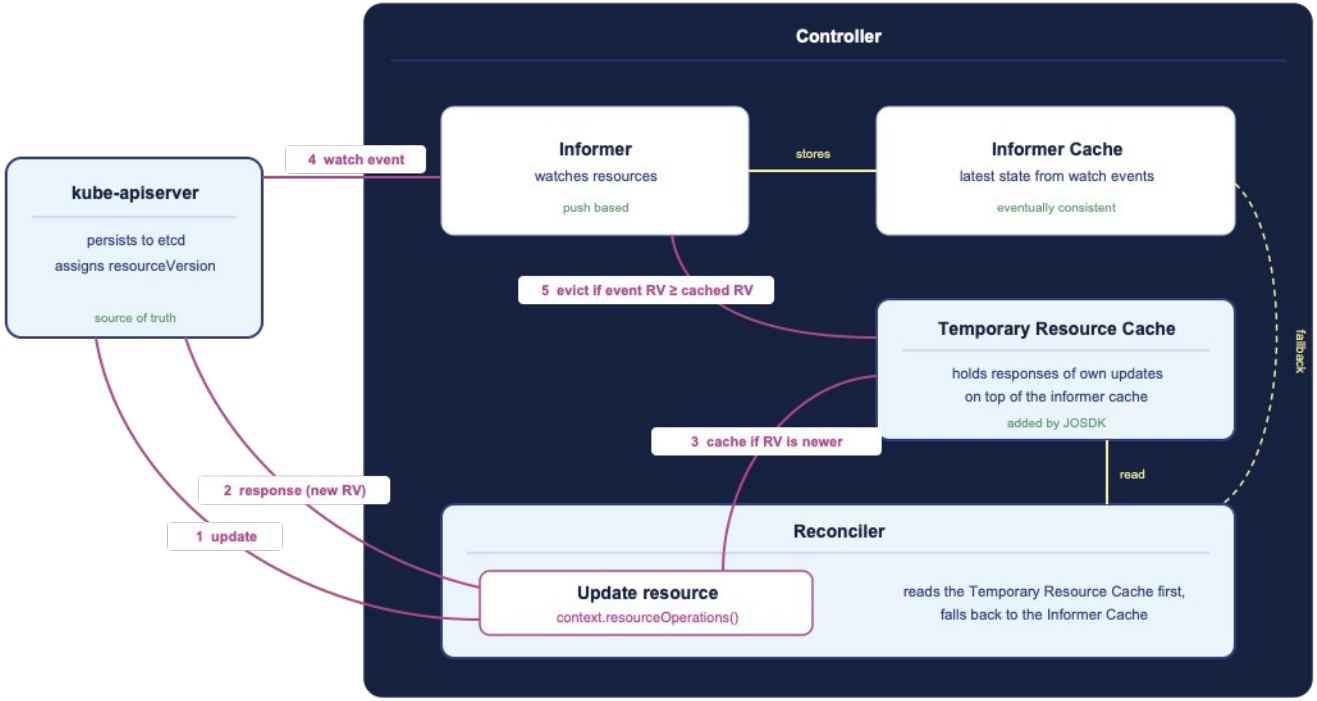


Operator Framework: Java Operator SDK

Read-Cache-After-Write Consistency

- since v5.3
- API:
  - UpdateControl.patchStatus(resource)
  - context.**resourceOperations()**.serverSideApply(resource);
- Additional cache brings complexity:
  - Indexers
  - Items store
- Comparable Resource Versions KEP
- Read-cache-after-write consistency blog

Read-Cache-After-Write Consistency

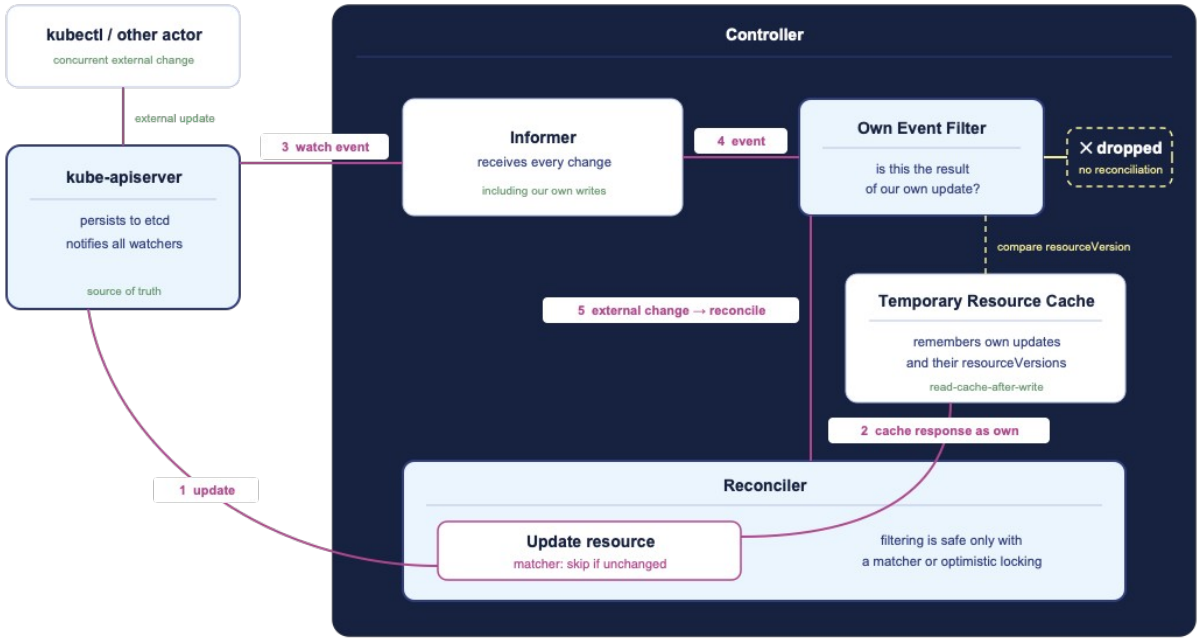


Event Filtering - API & Challenges

- Filter out events for our own updates
- Challenges
  a. Informer lost event problem
  b. Our update did not change the resource but other actor did concurrently
- context.**resourceOperations()**.serverSideApply(resource);

Event Filtering

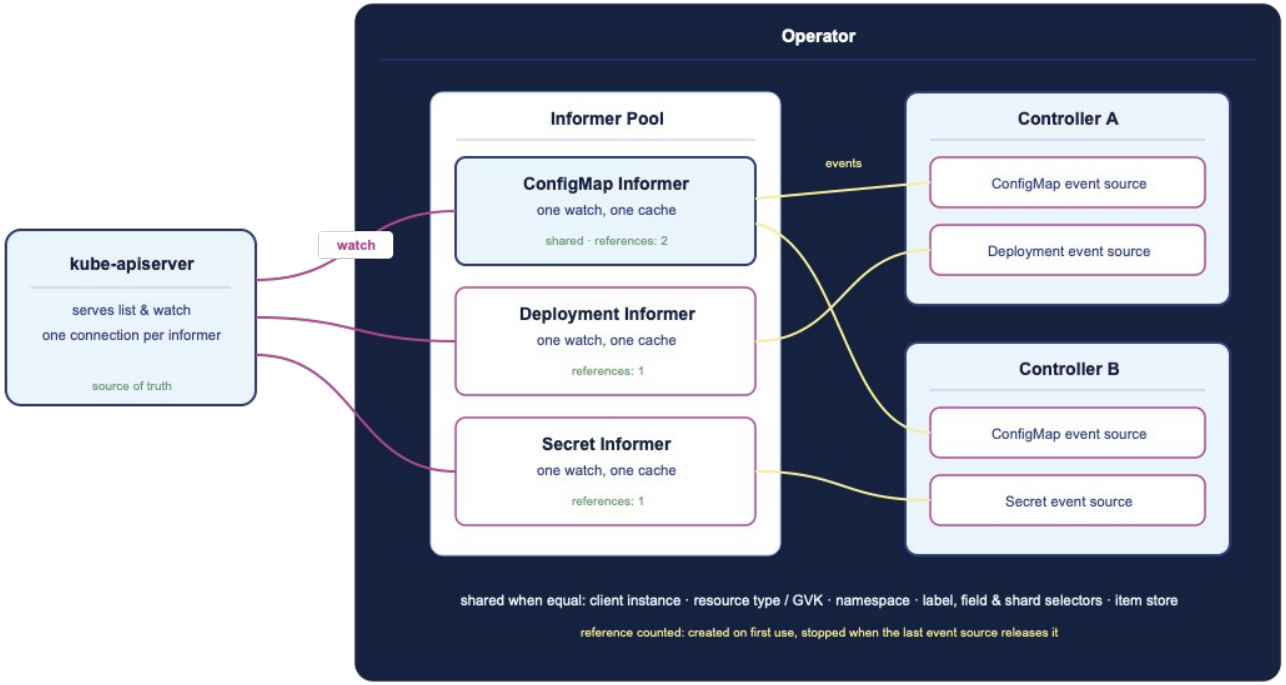
for Read-Cache-After-Write-Consistency



Informer Pools

- Sharing underlying informers between controller
- Matches the informer configuration
  - Except list limit
  - Details in InformerClassifier
- Supports dynamic Informer registration and de-registration

Informer Pools



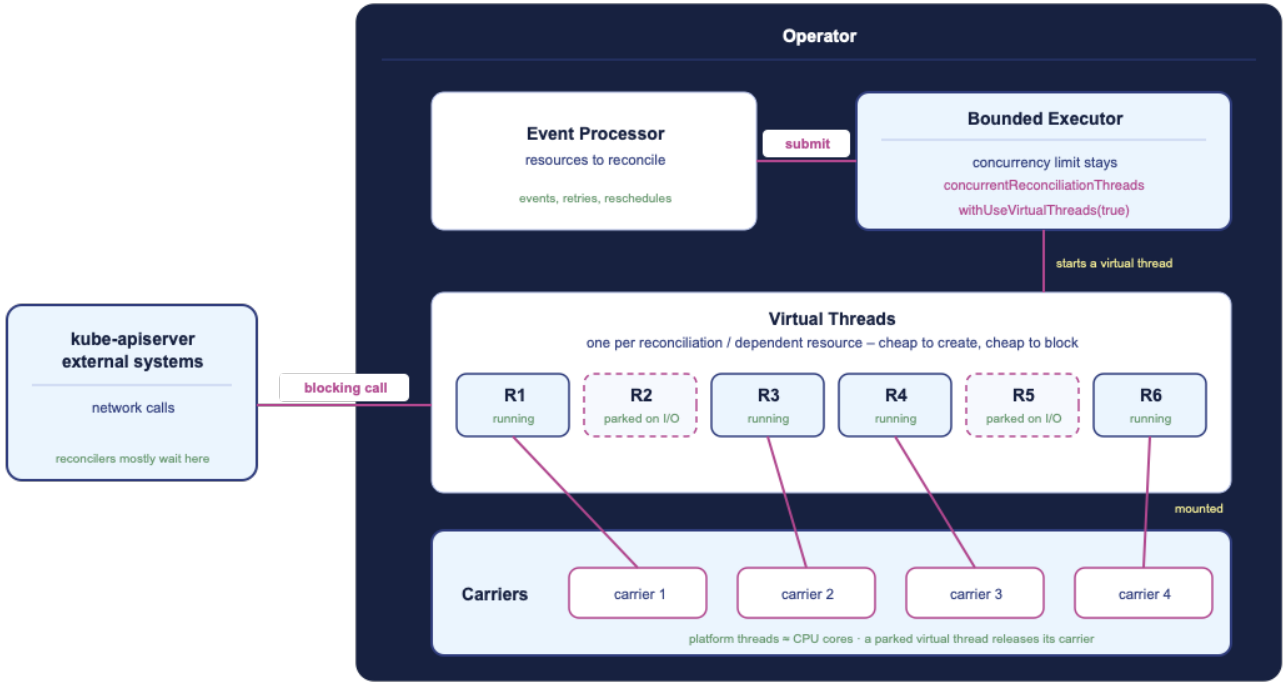
Observability stack enhancements

- Facelift over Observability stack
- Improved logging and Mapped Diagnostic Context (MDC)
  - Putting event and primary resource to MDC if available
- Metrics API refined
  - MicrometerMetricsV2
  - Generic Grafana Dashboard
  - Sample with OTel + Prometheus + Grafana

Java Virtual Threads (WIP)

- Planned for v5.7.0
- We will officially support this for Java 25+ (but will work with Java 21+)
  - Replacing ThreadPools with virtual threads across framework
- Virtually unlimited number of concurrent reconciliation
  - The explicit limit applies though

Java Virtual Threads (WIP)



Some honorable mentions…

- EventRecorder API
- ShardSelector support
- Documentation improvements - operations section

Resources and contribution steps

RESOURCE PACKAGE TO PREPARE

Use documented contracts and reproducible cases to guide contributions.

- Check content against a documented contract
- Share a reproducible case or fixture
- Select a maintainer-scoped contribution

Companion examples and task links are being prepared.

Contribution routes



Usage, issue reports, pull requests

Questions?

Questions about lifecycle APIs, packaging or controller behavior?

- OLM configuration and migration
- Helm content and catalog schemas
- Java Operator SDK reconciliation



Discussion of capabilities and proposed work.